

Sistema Distribuido sobre VPN WireGuard

Implementación de Algoritmos Distribuidos con Rust y Docker

 Equipo Rogelio (Hub)

 UDG - CUCEI

 Feb 2026

Backlog Sprint 1

Infraestructura Base

Objetivo: Dejar lista la red VPN y los contenedores para ejecutar el sistema distribuido.



- Configuración de VMs: Instalación de Linux (Arch o WSL) en los 4 nodos.



- Instalación de herramientas Docker, Docker Compose, iperf3, curl, wireguard,



- Levantar la VPN, configurar peers en Hub, y Spokers.



- Desplegar 16 contenedores "Workers" levantados (4 por integrante).



- Pruebas de conexión exitosa y performance



- Diseñar la implementación del algoritmo con Rust

Próximos Pasos (Sprint 2):



Programar algoritmo de Mandelbrot.



Optimizar velocidad de red.



Grabar video demostrativo.

Evidencia técnica del proyecto

wg show desde el Hub

```
[rogeliovm@archlinux proyecto-distribuidos]$ sudo wg show
interface: wg0
  public key: NK3tGMGpPvjhFjbrNEGtinhMn/lEFp5V0roL4N+iNBw=
  private key: (hidden)
  listening port: 51820

peer: MzeqrUWkSBuHEs1jG5u6io665zzAiV0b0KDexkbMji4=
  endpoint: 177.241.54.83:10558
  allowed ips: 10.10.10.2/32
  latest handshake: 7 seconds ago
  transfer: 48.06 KiB received, 40.58 KiB sent

peer: 1jEGvVWHzCxcF+dxDvfU8RgPisgP5K+VCe2bvh9qqyc=
  endpoint: 187.188.64.60:54196
  allowed ips: 10.10.10.3/32
  latest handshake: 17 seconds ago
  transfer: 180 B received, 92 B sent

peer: 100g58MT4lS8z4Ixpm++0eYWvcuV0p/d44i1uDkDsEY=
  endpoint: 177.231.15.207:14685
  allowed ips: 10.10.10.4/32
  latest handshake: 5 minutes, 19 seconds ago
  transfer: 16.54 KiB received, 13.64 KiB sent
[rogeliovm@archlinux proyecto-distribuidos]$
```

Tráfico Bidireccional:
Visibilidad Spoke-to-Spoke
gracias a la máscara /24 en
el Hub.

Estructura actual de la infraestructura

> 📡 Modelo Hub-and-Spoke

Arquitectura centralizada donde un nodo Maestro (Hub) orquesta y distribuye tareas a nodos periféricos (Spokes/Workers).

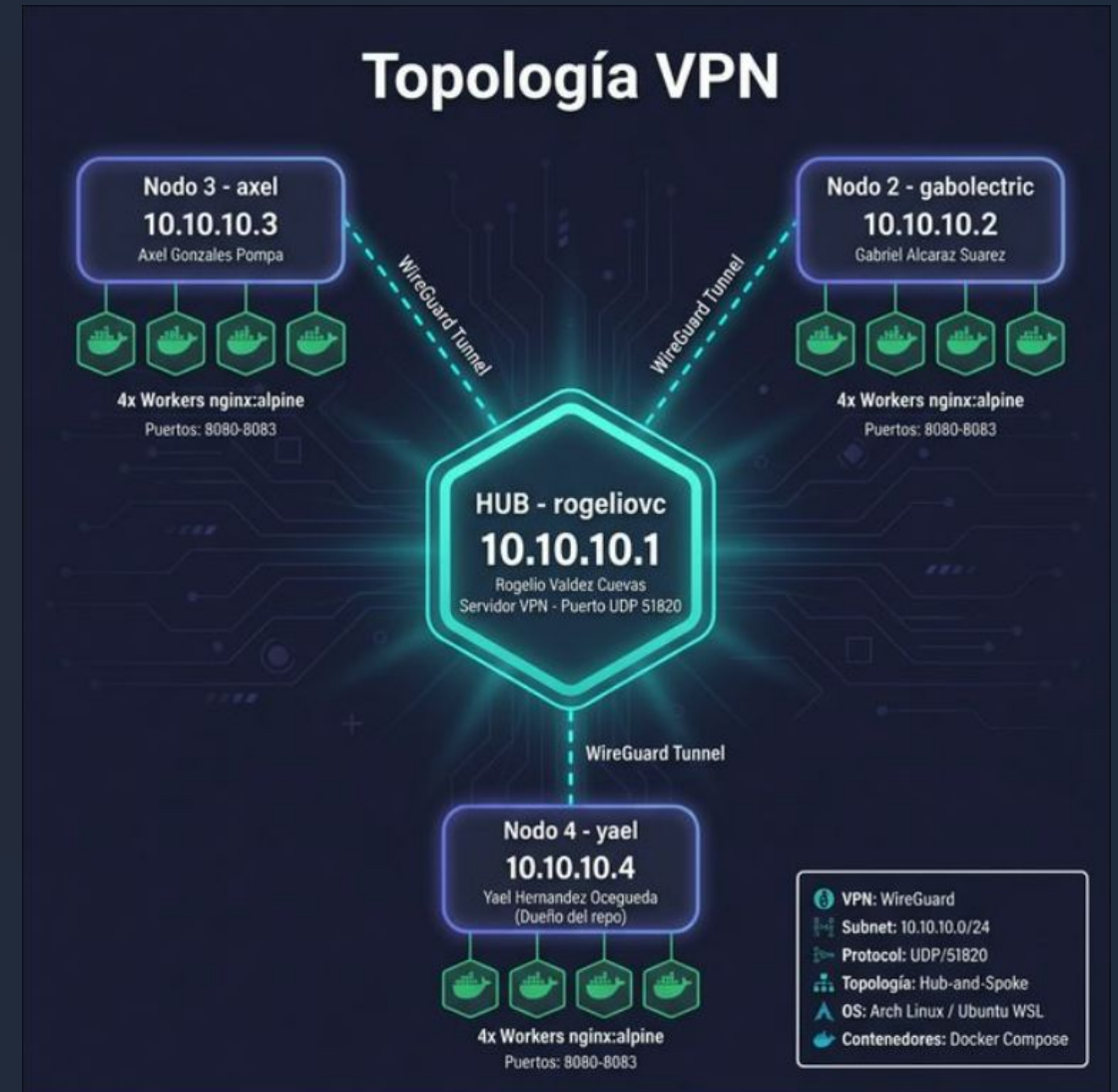
> 📦 Componentes

Hub (Rogelio): Gestiona ruteo y asignación.

Spokes (Gabriel, Axel, Yael): Ejecutan procesamiento intensivo.

> 🐧 Entorno Híbrido

Combinación de Arch Linux(QEMU/KVM), Ubuntu y Contenedores.



Metodología Basada en Scrum

Adoptamos un enfoque ágil inspirado en Scrum para organizar el desarrollo del proyecto.

El trabajo se dividió en Sprints con objetivos claros y entregables funcionales:

Sprint 1: Infraestructura (VPN + Docker + Comunicación TCP).

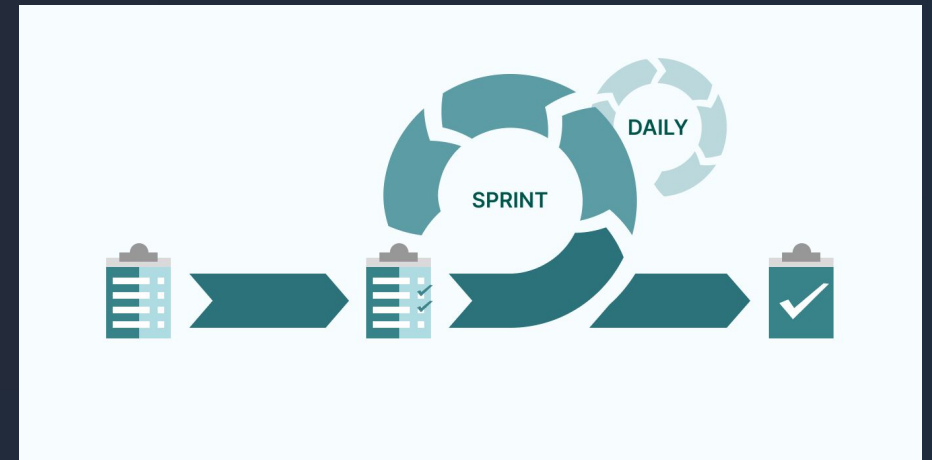
Sprint 2: Implementación del algoritmo distribuido (Mandelbrot).

Este enfoque permitió:

Reducir riesgos técnicos.

Detectar errores tempranamente.

Garantizar estabilidad antes de escalar el sistema.



Evidencia técnica del proyecto

Hub y Contenedores

```
(rogeliovm) localhost:2222 — Konsole
```

Nueva pestaña Dividir vista Copiar Pegar Buscar...

```
[rogeliovm@archlinux proyecto-distribuidos]$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
d53029e8da96	nginx:alpine	"/docker-entrypoint..."	42 minutes ago	Up 42 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	proyecto-distribuidos-worker-3
159b26ba03e8	nginx:alpine	"/docker-entrypoint..."	42 minutes ago	Up 42 minutes	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp	proyecto-distribuidos-worker-4
dca33c36ba7e	nginx:alpine	"/docker-entrypoint..."	42 minutes ago	Up 42 minutes	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp	proyecto-distribuidos-worker-2
22188c35597d	nginx:alpine	"/docker-entrypoint..."	42 minutes ago	Up 42 minutes	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp	proyecto-distribuidos-worker-1

```
[rogeliovm@archlinux proyecto-distribuidos]$
```

```
gaboLectric# sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
772b6ed297db	nginx:alpine	"/docker-entrypoint..."	38 seconds ago	Up 37 seconds	10.10.10.2:8080->80/tcp	docker-worker-1
5c1b60ff98bc	nginx:alpine	"/docker-entrypoint..."	38 seconds ago	Up 37 seconds	10.10.10.2:8081->80/tcp	docker-worker-3
b68f396d47c5	nginx:alpine	"/docker-entrypoint..."	38 seconds ago	Up 37 seconds	10.10.10.2:8082->80/tcp	docker-worker-4
3d04a4efd82c	nginx:alpine	"/docker-entrypoint..."	38 seconds ago	Up 37 seconds	10.10.10.2:8083->80/tcp	docker-worker-2

```
gaboLectric#
```

Evidencia técnica del proyecto

Contenedores

```
axel_@Pomposauria:~/los-dockerinos/DOCKER$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
2fce7c90f07c	nginx:alpine	"/docker-entrypoint..."	13 minutes ago	Up 15 seconds	10.10.10.3:8083->80/tcp	docker_worker_1
7f69deb72f9a	nginx:alpine	"/docker-entrypoint..."	13 minutes ago	Up 15 seconds	10.10.10.3:8080->80/tcp	docker_worker_4
b7869dcdd4db	nginx:alpine	"/docker-entrypoint..."	13 minutes ago	Up 15 seconds	10.10.10.3:8081->80/tcp	docker_worker_2
1ca0d7b00f3c	nginx:alpine	"/docker-entrypoint..."	13 minutes ago	Up 15 seconds	10.10.10.3:8082->80/tcp	docker_worker_3

```
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ sudo docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
0df5f71d5f07	nginx:alpine	"/docker-entrypoint..."	14 minutes ago	Up 14 minutes	10.10.10.4:8080->80/tcp	docker_worker_3
efcca774bf6b	nginx:alpine	"/docker-entrypoint..."	14 minutes ago	Up 14 minutes	10.10.10.4:8083->80/tcp	docker_worker_4
7da125cc8e5c	nginx:alpine	"/docker-entrypoint..."	14 minutes ago	Up 14 minutes	10.10.10.4:8082->80/tcp	docker_worker_2
44976e660b8c	nginx:alpine	"/docker-entrypoint..."	14 minutes ago	Up 14 minutes	10.10.10.4:8081->80/tcp	docker_worker_1
3dff97498be1	proyecto-distribuido_worker	"/target/release/wo..."	2 days ago	Up About an hour		proyecto-distribuido_worker_3
c66e1481381f	proyecto-distribuido_worker	"/target/release/wo..."	2 days ago	Up About an hour		proyecto-distribuido_worker_4
64b20393d6f9	proyecto-distribuido_worker	"/target/release/wo..."	2 days ago	Up About an hour		proyecto-distribuido_worker_1
95ec298cba90	proyecto-distribuido_worker	"/target/release/wo..."	2 days ago	Up About an hour		proyecto-distribuido_worker_2

```
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$
```


Evidencia técnica del proyecto

Pruebas entre nodos

```
gaboLectric# ping 10.10.10.4
PING 10.10.10.4 (10.10.10.4) 56(84) bytes of data.
64 bytes from 10.10.10.4: icmp_seq=1 ttl=63 time=52.2 ms
64 bytes from 10.10.10.4: icmp_seq=2 ttl=63 time=53.5 ms
64 bytes from 10.10.10.4: icmp_seq=3 ttl=63 time=58.7 ms
64 bytes from 10.10.10.4: icmp_seq=4 ttl=63 time=57.3 ms
64 bytes from 10.10.10.4: icmp_seq=5 ttl=63 time=50.3 ms
64 bytes from 10.10.10.4: icmp_seq=6 ttl=63 time=200 ms

--- 10.10.10.4 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5008ms
rtt min/avg/max/mdev = 50.271/78.702/200.186/54.404 ms
gaboLectric# |
```

```
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ ping 10.10.10.2
PING 10.10.10.2 (10.10.10.2) 56(84) bytes of data.
64 bytes from 10.10.10.2: icmp_seq=2 ttl=63 time=94.1 ms
64 bytes from 10.10.10.2: icmp_seq=3 ttl=63 time=59.9 ms
64 bytes from 10.10.10.2: icmp_seq=4 ttl=63 time=62.4 ms
64 bytes from 10.10.10.2: icmp_seq=5 ttl=63 time=55.3 ms
64 bytes from 10.10.10.2: icmp_seq=6 ttl=63 time=68.6 ms
64 bytes from 10.10.10.2: icmp_seq=7 ttl=63 time=54.7 ms
64 bytes from 10.10.10.2: icmp_seq=8 ttl=63 time=79.9 ms
64 bytes from 10.10.10.2: icmp_seq=9 ttl=63 time=62.4 ms
64 bytes from 10.10.10.2: icmp_seq=10 ttl=63 time=75.1 ms
64 bytes from 10.10.10.2: icmp_seq=11 ttl=63 time=62.7 ms
^C
--- 10.10.10.2 ping statistics ---
11 packets transmitted, 10 received, 9.09091% packet loss, time 10001ms
rtt min/avg/max/mdev = 54.684/67.503/94.132/11.705 ms
yael@DESKTOP-1B05468:/proyecto2/los-dockerinos/DOCKER$ |
```


Evidencia de

```
axel_@Pomposauria:~/los-dockerinos/DOCKER$ sudo wg show
interface: wg0
  public key: 1jEGvVWHzCxcF+dxDvfU8RgPiSgP5K+VCe2bvh9qqyc=
  private key: (hidden)
  listening port: 54402

peer: NK3tGMGpPvjhFjBRNEGtinhMn/LEFp5V0roL4N+iNBw=
  endpoint: 187.201.14.25:51820
  allowed ips: 10.10.10.0/24
  latest handshake: 5 seconds ago
  transfer: 92 B received, 180 B sent
  persistent keepalive: every 25 seconds
```

Handshake Exitoso: Confirmación de
conexión Peer-to-Peer mediante wg show.

Evidencia técnica del proyecto

Pruebas de performance

```
[rogeliovm@archlinux ~]$ iperf3 -s
-----
Server listening on 5201 (test #1)
-----
Accepted connection from 10.10.10.3, port 57364
[ 5] local 10.10.10.1 port 5201 connected to 10.10.10.3 port 57380
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-1.00    sec  0.00 Bytes  0.00 bits/sec
[ 5] 1.00-2.00    sec  640 KBytes  5.25 Mbits/sec
[ 5] 2.00-3.00    sec  896 KBytes  7.34 Mbits/sec
[ 5] 3.00-4.00    sec  0.00 Bytes  0.00 bits/sec
[ 5] 4.00-5.00    sec  1.75 MBytes 14.7 Mbits/sec
[ 5] 5.00-6.00    sec  2.25 MBytes 18.9 Mbits/sec
[ 5] 6.00-7.00    sec  2.00 MBytes 16.8 Mbits/sec
[ 5] 7.00-8.00    sec  2.38 MBytes 19.9 Mbits/sec
[ 5] 8.00-9.00    sec  2.50 MBytes 21.0 Mbits/sec
[ 5] 9.00-9.85    sec  2.12 MBytes 21.0 Mbits/sec
-----
[ ID] Interval      Transfer    Bitrate
[ 5] 0.00-9.85    sec  14.5 MBytes 12.3 Mbits/sec
```

receiver

```
axel_@Pomposauria:~$ iperf3 -c 10.10.10.1
Connecting to host 10.10.10.1, port 5201
[ 5] local 10.10.10.3 port 57380 connected to 10.10.10.1 port 5201
[ ID] Interval      Transfer    Bitrate    Retr  Cwnd
[ 5] 0.00-1.00    sec  768 KBytes  6.27 Mbits/sec  1    188 KBytes
[ 5] 1.00-2.00    sec  2.25 MBytes 18.9 Mbits/sec  0    754 KBytes
[ 5] 2.00-3.00    sec  0.00 Bytes  0.00 bits/sec  1    764 KBytes
[ 5] 3.00-4.00    sec  1.00 MBytes  8.40 Mbits/sec  6    550 KBytes
[ 5] 4.00-5.00    sec  1.88 MBytes 15.7 Mbits/sec 16    563 KBytes
[ 5] 5.00-6.00    sec  1.88 MBytes 15.7 Mbits/sec 58    304 KBytes
[ 5] 6.00-7.00    sec  2.00 MBytes 16.8 Mbits/sec  0    328 KBytes
[ 5] 7.00-8.00    sec  1.88 MBytes 15.7 Mbits/sec  0    344 KBytes
[ 5] 8.00-9.00    sec  1.88 MBytes 15.7 Mbits/sec  0    351 KBytes
[ 5] 9.00-10.05   sec  2.75 MBytes 22.1 Mbits/sec  0    352 KBytes
-----
[ ID] Interval      Transfer    Bitrate    Retr
[ 5] 0.00-10.05   sec  16.2 MBytes 13.6 Mbits/sec  82
[ 5] 0.00-9.85    sec  14.5 MBytes 12.3 Mbits/sec

iperf Done.
axel_@Pomposauria:~$
```

sender
receiver

Infraestructura de Contenedores



Docker Compose

Estandarización del entorno de ejecución. Garantiza que bibliotecas críticas como glibc sean consistentes entre Arch Linux, Ubuntu en nuestro caso



Network Mode: Host

Configuración clave:
`network_mode: "host"`. Permite a los contenedores usar directamente la interfaz virtual `wg0` del host sin capas NAT adicionales.



Microservicios

Despliegue aislado del binario `rust-worker`, asegurando inmutabilidad y fácil replicación en nuevos nodos (Spokes).

Algoritmo en Rust

Generación del Fractal de Mandelbrot

El problema computacional consiste en determinar la convergencia de una sucesión en el plano complejo mediante la iteración de la función $z_{n+1} = z_n^2 + c$.

Estrategia de División



Hub (Coordinador): Actúa como el orquestador del sistema. Divide el plano complejo en "chunks" o bloques de píxeles y los asigna dinámicamente a través de una API REST (Axum).



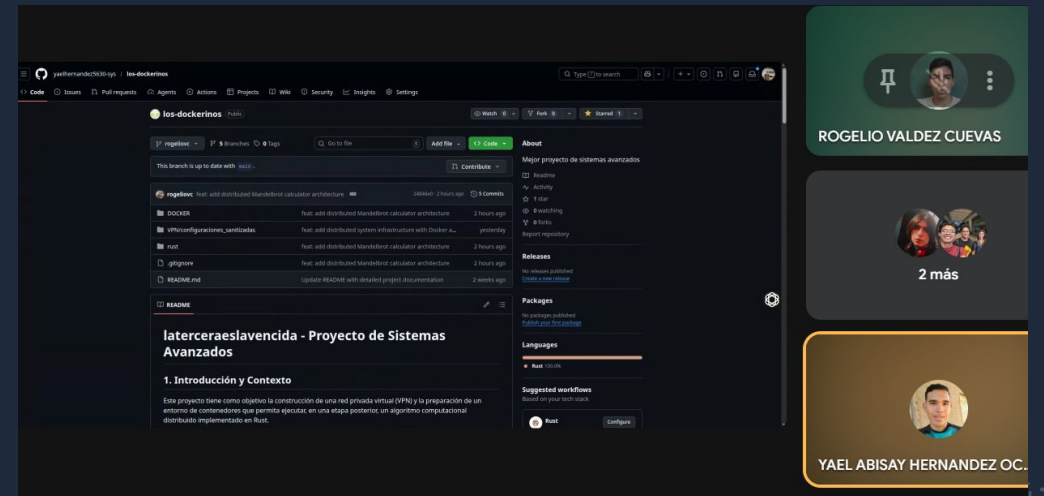
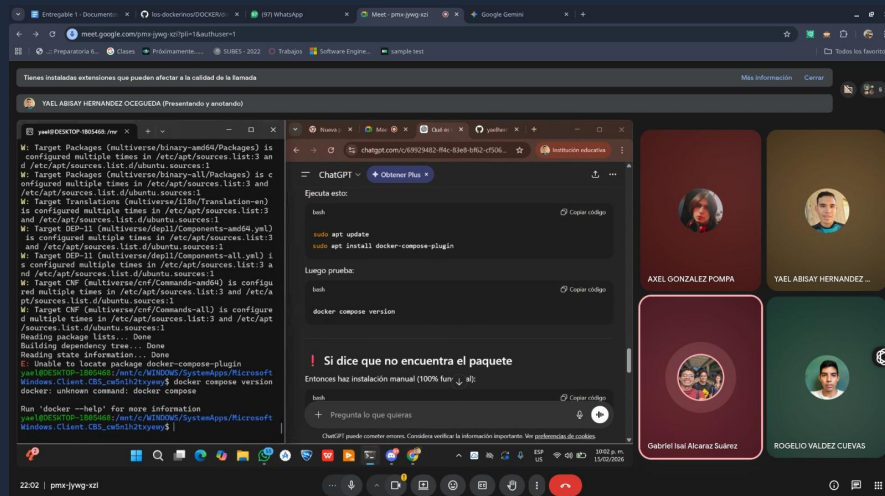
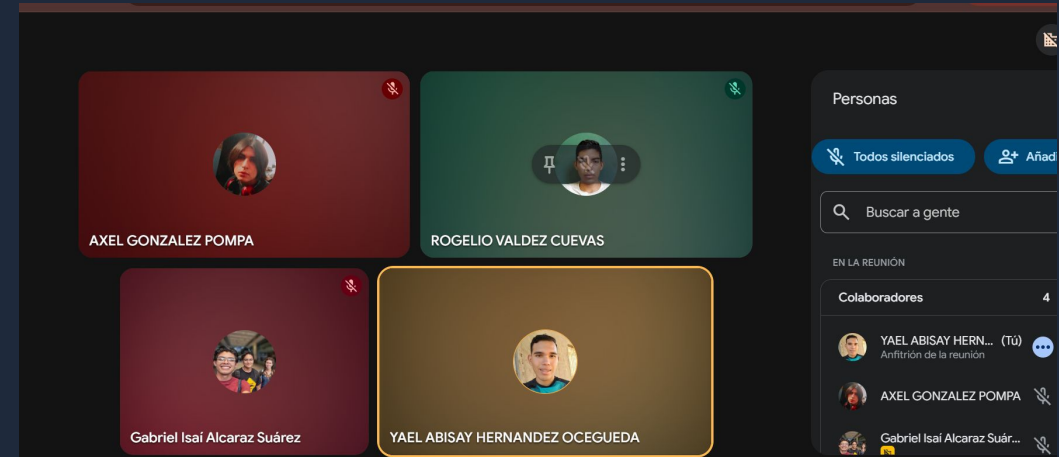
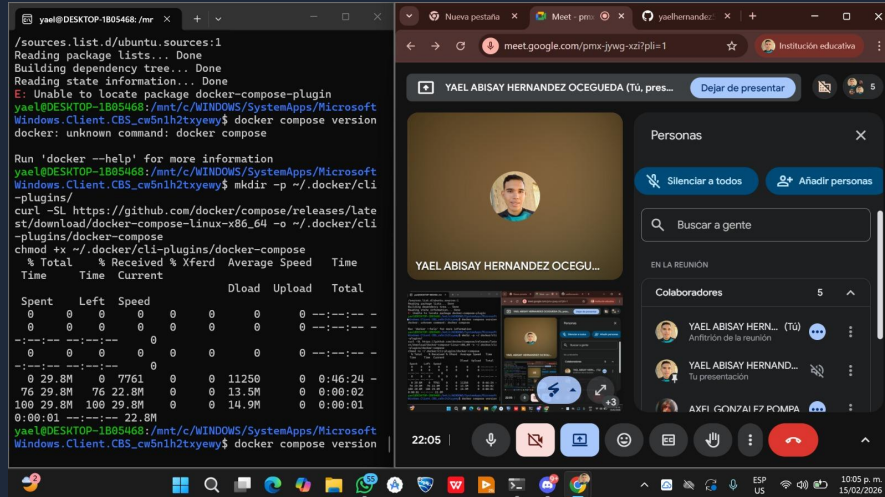
Workers (Nodos Docker): Cada contenedor solicita un bloque de trabajo, ejecuta el algoritmo de escape de tiempo en paralelo y reporta el estado de finalización al Hub.



¿Por qué Rust? Seguridad de memoria sin Garbage Collector y concurrencia eficiente.

```
1 mod models;
2 mod coordinator;
3 mod worker;
4
5 use std::env;
6
7 #[tokio::main]
8 async fn main() {
9     let args: Vec<String> = env::args().collect();
10
11     if args.len() < 2 {
12         println!("Use: cargo run -- [coordinator | worker <HUE_IP> <WORKER_ID>]");
13         return;
14     }
15
16     match args[1].as_str() {
17         "coordinator" => {
18             coordinator::run_coordinator().await;
19         }
20         "worker" => {
21             if args.len() < 4 {
22                 println!("Faltan argumentos. Uso: cargo run -- worker 10.20.10.1 Worker-Gabriel");
23                 return;
24             }
25             worker::run_worker(&args[2], &args[3]).await;
26         }
27         _ => println!("Comando no reconocido."),
28     }
29 }
```


Evidencia de Reuniones



Metodologia

ID	Épica	Tipo	Prioridad	Asignado_A	Título_de_la_Tarea	Descripción_y_Justificación	Criterios_de_Aceptación	Puntos	Estado	Comentarios_de_Cierre	Fecha
SCRUM-01	Gestión_Ágil	Ceremonia	Alta	Todos	Sprint Planning y Arquitectura	Definir la arquitectura Master-Worker, topología VPN, roles del equipo y elegir Rust por su eficiencia sin Garbage Collector.	Documento de arquitectura base, stack tecnológico (Máquinas virtuales, Docker, WireGuard) y división de tareas.	5	Finalizado	Sincronización inicial. Se acordó Sprint de 2 semanas con entregable 100% listo para etapa 1.	26/Ene - 28/Ene
INFRA-03	Infraestructura_Base	Historia	Critica	Todos	Levantamiento de VM Arch Linux (QEMU), y WSL para Windows	Como Sysadmin, necesito un entorno Linux aislado (Host) para orquestar la VPN y Docker sin afectar mi SO principal.	VM instalada y acceso SSH configurado	5	Finalizado	Se asignaron 8GB iniciales de disco virtual (.qcow2), lo cual requirió escalado posterior.	29/Ene - 30/Ene
INFRA-04	Infraestructura_Base	Historia	Alta	Todos		Preparar Arch Linux con las herramientas de containerización para el despliegue de los workers simulados.	Demon Docker activo (systemctl enable docker) y docker-compose instalado.	3	Finalizado	Permisos de grupo configurados para evitar uso excesivo de root en el build.	29/Ene - 30/Ene
NET-07	Red_VPN_y_Seguridad	Tarea	Alta	Todos	Generación de Perfiles y Llaves Peer (Privatekey y Publickey)	Crear los pares criptográficos para los workers remotos y enlazarlos al Hub.	3 archivos .conf generados con IPs 10.10.10.2 al 10.10.10.4.	3	Finalizado	Perfiles distribuidos de forma segura al resto del equipo.	30/Ene - 31/Ene
NET-05	Red_VPN_y_Seguridad	Historia	Critica	Rogelio	Configuración de Firewall y Port Forwarding	Para permitir conexiones externas a la VPN, se debe redirigir el tráfico del módem ISP hacia la VM local.	Puerto UDP 51820 abierto en módem y reglas de iptables/ufw permitiendo tráfico entrante.	8	Finalizado	Paso crítico de redes físicas completado. El Hub ahora es visible desde internet.	31/Ene - 1/Feb
NET-06	Red_VPN_y_Seguridad	Historia	Critica	Rogelio	Despliegue de Servidor WireGuard (Hub)	Establecer la interfaz wg0 en la VM para actuar como nodo orquestador central.	Servidor asignado a la IP estática 10.10.10.1 enrutando la subred.	5	Finalizado	Túnel maestro levantado exitosamente. Llaves del servidor generadas.	31/Ene - 1/Feb
NET-08	Red_VPN_y_Seguridad	Historia	Alta	Todos	Conexión de Peers Físicos Remotos	Instalar cliente WireGuard en equipos físicos e inyectar configuración para unirse al clúster privado.	Ping exitoso (<50ms) desde los 3 equipos hacia el 10.10.10.1.	5	Finalizado	Topología en estrella completada. Red privada virtual operativa.	2/Feb - 6/Feb
NET-09	Red_VPN_y_Seguridad	Ceremonia	Critica	Todos	Pruebas de conexión entre peers, hub y contenedores de todos los usuarios	Conectarse a una reunión para realizar pruebas en conjunto hacia todos las ip, y direcciones conocidas	Curl a contenedores usando curl http://10.10.10.x:808x	5	Finalizado	Topología en estrella completada. Red privada virtual operativa.	2/Feb - 6/Feb
DEV-11	Backend_Distribuido	Historia	Critica	Rogelio	Desarrollo API del Hub	Crear el orquestador HTTP que asigne 'chunks' de trabajo y reciba respuestas concurrentes.	Endpoints GET /task y POST /result activos en puerto 3000 de la VPN.	8	Finalizado	Manejo seguro de concurrencia. El Hub gestiona conexiones de múltiples peers a la vez.	7/Feb - 10/Feb
OPS-12	Orquestación_Docker	Historia	Alta	Yael	Creación de Dockerfile Multi-Etapa	Empaquetar el worker en una imagen ultra-ligera usando rust:alpine (compilación) y alpine:latest (ejecución).	Imagen de producción pesando menos de 50MB.	5	Finalizado	Base sólida creada, lista para escalado.	10/Feb - 12/Feb
BUG-13	Orquestación_Docker	Defecto	Critica	Rogelio	[BLOQUEO] Colapso de almacenamiento en VM	El entorno Arch Linux se quedó sin espacio ('no space left on device') por la caché pesada de compilación de Rust y Docker.	Sistema de archivos recuperado con al menos 15GB libres.	8	Resuelto	Expansión de disco QEMU a 20GB, re-partición con cfdisk y resize2fs en caliente. Operación de salvamento exitosa.	12/Feb - 14/Feb
BUG-14	Orquestación_Docker	Defecto	Alta	Rogelio	[BLOQUEO] Fallo DNS interno en Alpine Linux	Durante el build, los contenedores perdían internet al instalar utilidades ('transient error'), rompiendo la construcción.	Dockerfile compilando sin comandos 'apk add'.	3	Resuelto	Se aprovechó el caché del binario estático previo, eliminando la dependencia de red externa durante la creación de la imagen.	14/Feb - 14/Feb
OPS-15	Orquestación_Docker	Tarea	Alta	Rogelio	Despliegue y Escalado Local (Compose)	Crear docker-compose.yml para simular 4 workers locales conectados simultáneamente al Hub.	4 contenedores levantados mediante 'network_mode: host'.	5	Finalizado	Evade el NAT interno de Docker para que los contenedores hablen directamente por la interfaz wg0 (WireGuard).	12/Feb - 14/Feb
SCRUM-16	Gestión_Ágil	Ceremonia	Media	Todos	Sprint Review y Retrospective	Evaluación final del producto terminado frente a los requerimientos originales y documentación de lecciones aprendidas.	Documento final y reportes técnicos generados.	3	Finalizado	Lección principal: dimensionar correctamente el almacenamiento desde el Día 1 y gestionar la red NAT de Docker.	14/Feb - 15/Feb

Backlog Sprint 2



Desarrollo del Programa (Rust)

Implementar el servidor (Coordinador) encargado de dividir, repartir y monitorear la carga de trabajo.



Contenerización y Orquestación (Docker)

Acoplar los contenedores directamente a la interfaz de la VPN conteniendo el código de Rust para asegurar una comunicación directa y rápida.



Pruebas y Resolución de Problemas

Ejecutar pruebas de carga reales con todos los nodos del equipo conectados al mismo tiempo.

Arquitectura Orientada a **Microservicios**



Coordinator (Axum)

Aplicación asíncrona centralizada. Expone una API REST, subdivide el trabajo matemáticamente y salvaguarda el progreso utilizando estado concurrente y mutabilidad controlada.



Workers (Request)

Nodos periféricos "tontos" pero de alta intensidad computacional. Solicitan tareas infinitamente (Polling), evalúan el algoritmo y devuelven arreglos de píxeles puros.



Serialización (Serde)

Intercambio estricto de JSON. Traduce la memoria de las estructuras `Task` y `TaskResult` a cadenas de red para viajar a través del túnel VPN.

El Coordinador: Memoria y Estado

Estado Concurrente

- La seguridad de datos compartidos es clave en hilos asíncronos.
- El estado del Coordinador usa `Arc<Mutex<T>>` para garantizar protección.
- Esto evita conflictos de datos al manejar múltiples solicitudes simultáneas de Workers.
- Asegura que las tareas sean solicitadas sin errores ni corrupción.

Particionamiento Dinámico

El Coordinador lee las dimensiones de la imagen desde las variables de entorno

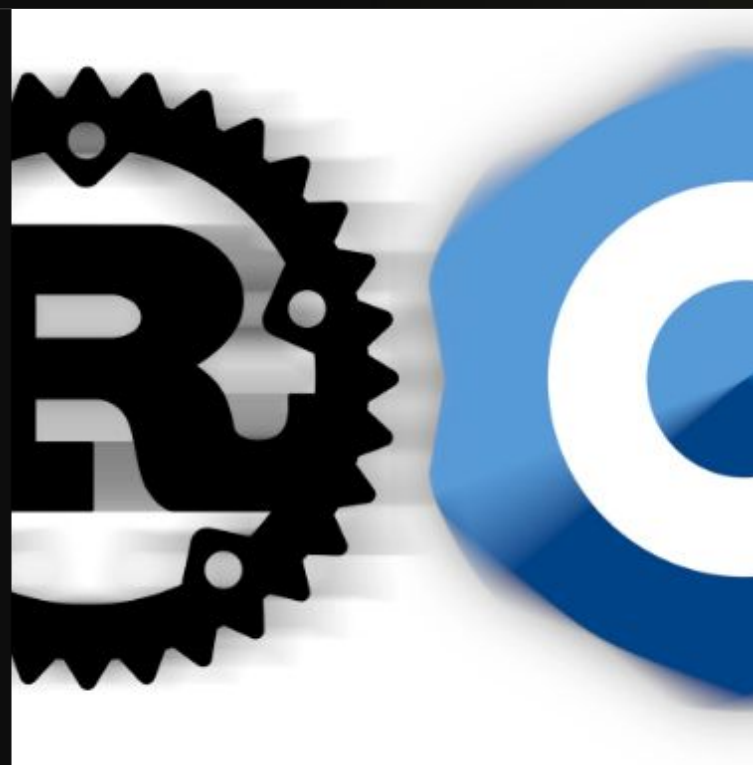
La carga no se procesa entera; se fragmenta en regiones horizontales de la imagen. La estructura `ImageRegion` se delega con su identificador propio.

Modelado Estricto de Datos

-  **MandelbrotParams:** Fija la resolución y los límites del plano complejo (coordenadas X/Y) para el cálculo del fractal.
-  **Task:** Contrato enviado al Worker. Incluye un `task_id` único y los límites de la sub-región a calcular (`y_start` a `y_end`).
-  **TaskResult:** El resultado procesado. Incluye métricas de rendimiento (`computation_time_ms`) y el vector crítico `Vec<u8>` que alberga el mapa de píxeles en escala de grises.
-  **Agnóstico al Host:** Gracias a `#[derive(Serialize, Deserialize)]` de Serde, cualquier nodo entiende y valida instantáneamente este formato JSON.

Ciclo de Vida de los **Workers**

- 🔄 **Polling Continuo:** Los Workers ejecutan un bucle infinito enviando peticiones GET a `/get_task`, simplificando la comunicación.
- 🛡️ **Resiliencia a Fallos:** Si el Hub no responde o la VPN colapsa, el patrón `match` de Rust captura el error y realiza una pausa controlada (`tokio::time::sleep`) antes de reintentar.
- 🧠 **Ejecución en Memoria Pura:** El cálculo se realiza directamente en la memoria y CPU del Worker. El resultado se empaqueta en bytes para su envío, asegurando que un Worker no interfiera con el trabajo de otro.



El Conjunto de Mandelbrot

256

Iteraciones Máximas por Coordenada

Algoritmo Fractal en Rust

Por cada coordenada de píxel, se determina si el punto equivalente en el plano complejo escapa al infinito, mediante una fórmula matemática que se aplica repetidamente.

$$Z_{n+1} = Z_n^2 + C$$

En código, esto se traduce a monitorear $x*x + y*y \leq 4.0$.

El número de iteraciones que se logran antes de cruzar el umbral define la intensidad o color (Luminancia 0-255) del píxel final.

Contenerización: Multi-Stage



Fase de Construcción (Builder): El código se compila dentro de un contenedor aislado

```
rust:alpine
```

 generando un binario estático (

```
--release
```

).

Fase de Despliegue (Runner): Se descarta toda la maquinaria de Rust y las librerías base. La imagen de producción finaliza alojada en un sistema operativo purgado (`alpine:latest`).



Peso Mosca: La imagen total desciende a ~50MB. Minimiza el consumo de red, memoria, y reduce drásticamente las vulnerabilidades y la superficie de ataque.



Despliegue Paralelo y Orquestación

Composición Declarativa

El uso de `docker-compose.yml` permite levantar 4 contenedores Worker idénticos por nodo físico, cada uno asignado con un id de entorno único

(`WORKER_ID`).

Garantizando que no existan desajustes de versiones de entre las distintas distribuciones Linux de los miembros del equipo.

Pila de Red Bypass

Aplicando `network_mode: "host"` instruimos a Docker a ignorar su red interna virtual (Bridge) y sus propias reglas NAT.

El contenedor captura de manera directa las tarjetas de red de la máquina huésped, comunicándose directamente por la interfaz `wg0` eliminando la latencia artificial.

Métricas y Resultados Finales

480k

Píxeles Renderizados

Calculados iteración tras iteración matemática de forma distribuida logrando una resolución base de 800x600 mapeada en memoria cruda.

16

Hilos Asíncronos

Desplegados transparentemente usando Docker. El Coordinador despachó y recibió confirmación de las 16 tareas segmentadas con precisión.

Cálculo y Ensamblaje

El Coordinador integró los vectores de 16 máquinas devolviendo el archivo PNG final en fracciones de segundo con el crate `image`.

Métricas y Resultados Finales

Dockers trabajando en conjunto

```

Attaching to rust_worker_1, rust_worker_2, rust_worker_3, rust_worker_4
rust_worker_2 | * Modo Worker
rust_worker_2 | 🔗 Conectando a: 10.10.10.1:3000
rust_worker_2 | 🔒 Worker ID: worker-gabolectric-2
rust_worker_2 | * Iniciando Worker worker-gabolectric-2... Conectando al Coordinator en 10.10.10.1:3000
rust_worker_4 | * Modo Worker
rust_worker_4 | 🔗 Conectando a: 10.10.10.1:3000
rust_worker_4 | 🔒 Worker ID: worker-gabolectric-4
rust_worker_4 | * Iniciando Worker worker-gabolectric-4... Conectando al Coordinator en 10.10.10.1:3000
rust_worker_3 | * Modo Worker
rust_worker_3 | 🔗 Conectando a: 10.10.10.1:3000
rust_worker_3 | 🔒 Worker ID: worker-gabolectric-3
rust_worker_3 | * Iniciando Worker worker-gabolectric-3... Conectando al Coordinator en 10.10.10.1:3000
rust_worker_1 | * Modo Worker
rust_worker_1 | 🔗 Conectando a: 10.10.10.1:3000
rust_worker_1 | 🔒 Worker ID: worker-gabolectric-1
rust_worker_1 | * Iniciando Worker worker-gabolectric-1... Conectando al Coordinator en 10.10.10.1:3000

rust_worker_3 | Worker worker-gabolectric-3: Recibió tarea 10 para región [0,800] x [333,370]
rust_worker_4 | Worker worker-gabolectric-4: Recibió tarea 14 para región [0,800] x [481,518]
rust_worker_2 | Worker worker-gabolectric-2: Recibió tarea 13 para región [0,800] x [444,481]
rust_worker_1 | Worker worker-gabolectric-1: Recibió tarea 12 para región [0,800] x [407,444]

rust_worker_2 | Worker worker-gabolectric-2: Cálculo completado en 12ms, 29600 pixeles generados
rust_worker_4 | Worker worker-gabolectric-4: Cálculo completado en 5ms, 29600 pixeles generados

rust_worker_3 | Worker worker-gabolectric-3: Cálculo completado en 25ms, 29600 pixeles generados
rust_worker_1 | Worker worker-gabolectric-1: Cálculo completado en 21ms, 29600 pixeles generados
rust_worker_2 | Worker worker-gabolectric-2: Resultado enviado al coordinator
rust_worker_4 | Worker worker-gabolectric-4: Resultado enviado al coordinator
rust_worker_1 | Worker worker-gabolectric-1: Resultado enviado al coordinator
rust_worker_3 | Worker worker-gabolectric-3: Resultado enviado al coordinator

```

```
rust_worker_3 | Worker worker-axel-3: Recibió tarea 7 para región [0,800] x [222,259]
rust_worker_4 | Worker worker-axel-4: Recibió tarea 6 para región [0,800] x [185,222]
rust_worker_2 | Worker worker-axel-2: Recibió tarea 5 para región [0,800] x [148,185]
rust_worker_1 | Worker worker-axel-1: Recibió tarea 8 para región [0,800] x [259,296]
rust_worker_2 | Worker worker-axel-2: Cálculo completado en 13ms, 29600 píxeles generados
rust_worker_4 | Worker worker-axel-4: Cálculo completado en 17ms, 29600 píxeles generados
rust_worker_3 | Worker worker-axel-3: Cálculo completado en 22ms, 29600 píxeles generados
rust_worker_1 | Worker worker-axel-1: Cálculo completado en 27ms, 29600 píxeles generados
rust_worker_2 | Worker worker-axel-2: Resultado enviado al coordinator
rust_worker_4 | Worker worker-axel-4: Resultado enviado al coordinator
rust_worker_3 | Worker worker-axel-3: Resultado enviado al coordinator
rust_worker_1 | Worker worker-axel-1: Resultado enviado al coordinator
rust_worker_3 | Worker worker-axel-3: Recibió tarea 9 para región [0,800] x [296,333]
rust_worker_2 | Worker worker-axel-2: Recibió tarea 10 para región [0,800] x [333,370]
rust_worker_1 | Worker worker-axel-1: Recibió tarea 8 para región [0,800] x [259,296]
rust_worker_4 | Worker worker-axel-4: Recibió tarea 11 para región [0,800] x [370,407]
rust_worker_4 | Worker worker-axel-4: Cálculo completado en 19ms, 29600 píxeles generados
rust_worker_2 | Worker worker-axel-2: Cálculo completado en 24ms, 29600 píxeles generados
rust_worker_1 | Worker worker-axel-1: Cálculo completado en 27ms, 29600 píxeles generados
rust_worker_3 | Worker worker-axel-3: Cálculo completado en 29ms, 29600 píxeles generados
rust_worker_4 | Worker worker-axel-4: Resultado enviado al coordinator
rust_worker_1 | Worker worker-axel-1: Resultado enviado al coordinator
rust_worker_2 | Worker worker-axel-2: Resultado enviado al coordinator
rust_worker_3 | Worker worker-axel-3: Resultado enviado al coordinator
```

Métricas y Resultados Finales

```
[rogeliovm@archlinux los-dockerinos]$ cd rust && ./target/release/mandelbrot_distribuido coordinator
📺 Modo Coordinator
🔧 Iniciando Coordinator...
📁 Coordinator: Generadas 16 tareas para imagen 800x600
🌐 Coordinator escuchando en 0.0.0.0:3000
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 16 (región [0,800] x [555,600])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 15 (región [0,800] x [518,555])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 14 (región [0,800] x [481,518])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 13 (región [0,800] x [444,481])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 12 (región [0,800] x [407,444])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 11 (región [0,800] x [370,407])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 10 (región [0,800] x [333,370])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 9 (región [0,800] x [296,333])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 8 (región [0,800] x [259,296])
👤 Coordinator: Recibido resultado de tarea 12 del worker worker-rogelio-1 (10ms, 29600 píxeles)
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 7 (región [0,800] x [222,259])
👤 Coordinator: Recibido resultado de tarea 15 del worker worker-gaboelectric-1 (0ms, 29600 píxeles)
👤 Coordinator: Recibido resultado de tarea 13 del worker worker-gaboelectric-3 (7ms, 29600 píxeles)
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 6 (región [0,800] x [185,222])
👤 Coordinator: Recibido resultado de tarea 7 del worker worker-rogelio-3 (36ms, 29600 píxeles)
👤 Coordinator: Recibido resultado de tarea 6 del worker worker-rogelio-4 (17ms, 29600 píxeles)
👤 Coordinator: Recibido resultado de tarea 14 del worker worker-gaboelectric-2 (7ms, 29600 píxeles)
👤 Coordinator: Recibido resultado de tarea 16 del worker worker-gaboelectric-4 (0ms, 36000 píxeles)
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 5 (región [0,800] x [148,185])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 4 (región [0,800] x [111,148])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 3 (región [0,800] x [74,111])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 2 (región [0,800] x [37,74])
📁 Coordinator: Recibida solicitud de tarea
👤 Coordinator: Asignando tarea 1 (región [0,800] x [0,37])
```

```
rust_worker_4 | 🔗 Conectando a: 10.10.10.1:3000

rust_worker_1 | 🆔 Worker ID: worker-yael-1
rust_worker_3 | 🆔 Worker ID: worker-yael-3

rust_worker_4 | 🆔 Worker ID: worker-yael-4

rust_worker_4 | ⚙️ Iniciando Worker worker-yael-4... Conectando al Coordinator en 10.10.10.1:3000
rust_worker_1 | ⚙️ Iniciando Worker worker-yael-1... Conectando al Coordinator en 10.10.10.1:3000
rust_worker_2 | 📁 Worker worker-yael-2: Recibió tarea 16 para región [0,800] x [555,600]
rust_worker_2 | 📁 Worker worker-yael-2: Cálculo completado en 2ms, 36000 píxeles generados

rust_worker_4 | 📁 Worker worker-yael-4: Recibió tarea 9 para región [0,800] x [296,333]
rust_worker_1 | 📁 Worker worker-yael-1: Recibió tarea 15 para región [0,800] x [518,555]

rust_worker_3 | 📁 Worker worker-yael-3: Recibió tarea 11 para región [0,800] x [370,407]

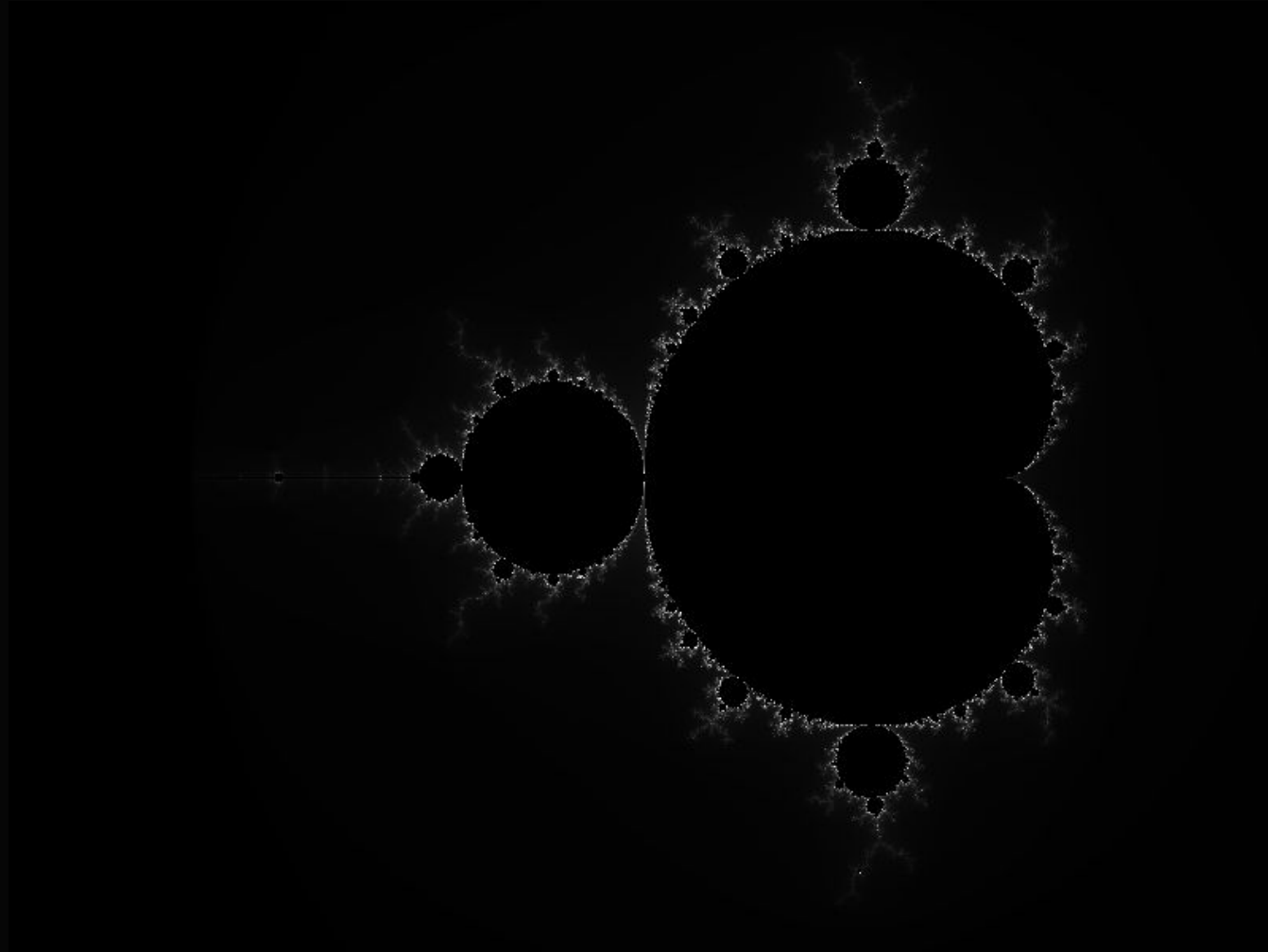
rust_worker_1 | 📁 Worker worker-yael-1: Cálculo completado en 2ms, 29600 píxeles generados
rust_worker_3 | 📁 Worker worker-yael-3: Cálculo completado en 12ms, 29600 píxeles generados
rust_worker_4 | 📁 Worker worker-yael-4: Cálculo completado en 17ms, 29600 píxeles generados
rust_worker_1 | 📁 Worker worker-yael-1: Resultado enviado al coordinator
rust_worker_3 | 📁 Worker worker-yael-3: Resultado enviado al coordinator

rust_worker_2 | 📁 Worker worker-yael-2: Resultado enviado al coordinator
rust_worker_4 | 📁 Worker worker-yael-4: Resultado enviado al coordinator
|
```

Métricas y Resultados Finales

```
Coordinator: Recibido resultado de tarea 11 del worker worker-axel-4 (19ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 5 del worker worker-rogelio-2 (8ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 8 del worker worker-axel-1 (27ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 10 del worker worker-axel-2 (24ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 9 del worker worker-axel-3 (29ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 1 del worker worker-yael-3 (0ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 2 del worker worker-yael-1 (0ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 3 del worker worker-yael-4 (1ms, 29600 píxeles)
Coordinator: Recibido resultado de tarea 4 del worker worker-yael-2 (2ms, 29600 píxeles)
Coordinator: ¡Todas las 16 tareas completadas! Iniciando ensamblaje...
Coordinator: Ensamblando imagen final...
Coordinator: Imagen final ensamblada:
- Total píxeles: 480000
- Tiempo total de cómputo: 187ms
- Tareas procesadas: 16/16
- Tiempo promedio por tarea: 11ms
```


Métricas y Resultados Finales



Metodologia

Tabla_1 												
	ID	Épica	Tipo	Prioridad	Asignado_A	Título_de_la_Tarea	Descripción_y_justificación	Criterios_de_Aceptación	Puntos	Estados	Comentarios_de_Cierre	Fecha
2	DEV-17	Backend_Distribuido	Historia	Crítica	Todos	Implementación de lógica Mandelbrot	Desarrollar la función iterativa $z_{n+1} = z_n^2/n + c$ para determinar la pertenencia al conjunto en el Worker.	Cálculo correcto de iteraciones y mapeo a escala de grises (0-255).	5	Finalizado		01/Mar - 03/Mar
3	DEV-18	Backend_Distribuido	Tarea	Alta	Rogelio	Protocolo de comunicación de HTTP	Definir los contratos JSON (model.rs) para el intercambio de tarea y resultados entre nodos.	Serialización/Deserialización exitosa con serde.	3	Finalizado		03/Mar - 04/Mar
4	OPS-19	Orquestación_Docker	Historia	Alta		Orquestación Multicontenedor	Crea el archivo docker-compose.yml para desplegar el Coordinator y múltiples workers con variables de entorno dinámicas.	Despliegue de 1 Coordinator y N Workers con un solo comando.	5	Finalizado		04/Mar - 06/Mar
5	QA-20	Calidad_y_Puebas	Ceremonia	Crítica	Todos	Prueba de Integración y Reparto	Reunión conjunta para validar la división de la imagen en 'chunks' y la recepción concurrente de resultados.	Imagen completa ensamblada en memoria sin pérdida de paquetes.	5	Finalizado		07/mar
6	DEV-21	Backend_Distribuido	Historia	Media		Persistencia de Imagen Final	Implementar el guardado de los píxeles ensamblados en un formato de imagen (PGM/PNG) tras completar todas las tareas.	Archivo de imagen generado físicamente en el volumen del Coordinator.	3	Finalizado		08/Mar - 09/Mar

Metodologia

Distributed Mandelbrot - Project Board

View 1

+ New view

Filter by keyword or by field

Todo

0

...

This item hasn't been started

+ Add item

In Progress

0

...

This is actively being worked on

+ Add item

Done

4

...

+

This has been completed

los-dockerinos #2

Prepare Docker for distributed Rust algorithm (mandelbrot) for everyone - Code ready in branch "rogeliovc"

los-dockerinos #3

Implement distributed Mandelbrot algorithm in Rust

los-dockerinos #4

Test distributed Mandelbrot execution and image assembly

los-dockerinos #5

...

Epic: Project Management and Documentation Overhaul

+ Add item